Beyond Linear Scaling for LLM Training on Wafer-Scale GPUs

MICRO 2026 Submission **#2250** – Confidential Draft – Do NOT Distribute!!

Abstract

Wafer-scale GPUs integrate many dies on a single substrate to break the reticle limit for LLM training. Current systems adopt distributed execution that achieves at best linear scaling, but linear scaling is suboptimal: ideal single-GPU performance scaling, e.g., attainable FLOP per second relative to the hardware resources, is beyond linear, as proportional resource scaling creates memory-system redundancy. The root cause preventing this is *L2 isolation*, where distributed execution confines each die to its local L2.

We propose Halo, a hardware-software co-design framework that breaks L2 isolation through architecturally managed SM-direct remote loads, enabling practical cross-die L2 sharing at wafer scale. *Remote-access scheduling* reduces communication overhead by confining traffic within topology-aligned regions and partitioning wafer into L2-sharing domains. *Speculative pipelining* hides the elevated remote-access latency by deferring shared-memory allocation until data arrives, allowing deep software pipelining beyond physical shared-memory capacity. Evaluated on a cycle-accurate simulator with diverse LLM workloads, Halo achieves 1.24× average speedup over state-of-the-art distributed execution, reaching 98% of the theoretical beyond-linear bound, at <0.02% die-area overhead.

1 Introduction

The rapid growth of large language models (LLMs) demands ever-growing computation, memory, and bandwidth from training systems [3, 4, 23, 45]. Yet scaling a monolithic GPU is increasingly constrained by the *reticle limit*, the field-size limitation of lithography steppers that caps a single die at 858 mm^2 [28]. State-of-the-art GPU dies such as NVIDIA H100 [29] and B200 [30] already approach this limit, leaving little room for single-die expansion. Recent progress in chiplet design [36], die-to-die interconnects [44], and advanced packaging [24] pushes chip scale-up beyond this barrier. As the ultimate extension, wafer-scale GPUs [21, 32] integrate many reticle-limited dies on a single substrate, offering higher compute density, greater inter-die bandwidth, and lower latency than conventional multi-GPU systems, making them a compelling platform for LLM training [41, 46, 47, 49, 53].

The most natural approach to a wafer-scale GPU is transparent **single-GPU execution** [32]: exposing the entire wafer as one large GPU so that performance scales without software awareness of the multi-die structure. However, as the die count grows to tens or more, global all-to-all accesses incur prohibitive inter-die communication overhead, reducing compute utilization to below 30%, as confirmed in Section 4.2.1. Therefore, current systems retreat to **distributed execution** [41, 46, 47, 49, 53], treating the wafer as a tightly integrated multi-GPU system. Through communication-centric optimizations such as collective operators and compute-communication fusion, distributed execution achieves near-linear scaling: When resources, including total die area and HBM bandwidth, increase to $N\times$, performance also improves by $N\times$. In the multi-GPU literature, linear scaling is widely regarded as optimal.

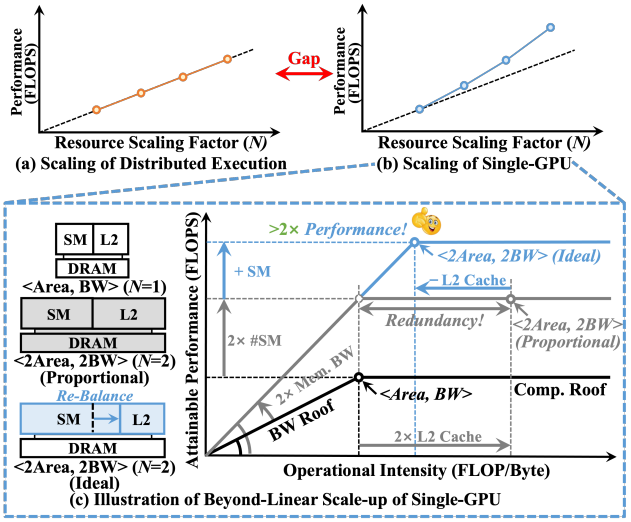


Figure 1: Gap between linear scaling and ideal beyond-linear scaling. (a) Distributed execution scales linearly. (b) An ideal single GPU scales beyond linear. (c) Roofline explanation of (b) for N=2: proportional resource scaling creates memory-system redundancy, while an ideal single GPU reallocates area of redundant L2 to SMs, achieving >2× performance.

However, we argue that linear scaling is actually a sub-optimal target: it is optimal when additional dies are treated as several independent GPU instances, but not when scaling up one ideal single GPU. An ideal single GPU should exhibit beyond-linear scaling, i.e., performance can improve by more than $N\times$ when resources scale up by $N\times$, as shown in Figure 1(b). Roofline model in Figure 1(c) explains why the ideal single-GPU scale-up in Figure 1(b) can be beyond linear, using an N=2 example. When die area and DRAM bandwidth are doubled, proportionally doubling SMs and L2 can have a 2× performance. However, the doubled L2 also raises operational intensity past the balance point, pushing the design into compute-bound regime and exposing *memory-system redundancy*. An ideal single GPU can therefore reduce redundant L2 capacity and reallocate freed area to additional SMs, increasing compute roof beyond the proportional 2× design and achieving *beyond-linear* scaling. This rebalance is illustrated as "−L2 Cache" and "+SM" relative to the proportional scaling point, yielding more than 2× performance.

The root cause of this gap between linear scaling of distributed execution and ideal performance scaling is *L2 isolation* induced by communication-centric philosophy of distributed execution. Exploiting a remote die's L2 for data reuse requires repeatedly accessing the same remote data, which directly conflicts with the goal of minimizing inter-die communication. Distributed execution therefore confines each die to its own local L2. This pins per-die operational intensity at the single-die balance point and prevents beyond-linear scaling even as total resources scale. On conventional multi-GPU systems, this isolation is inescapable: their limited inter-chip bandwidth cannot support repeated remote accesses that L2 sharing

demands. Wafer-scale GPUs, with their substantially richer inter-die fabric, create a new opportunity to break this isolation. Section 2 substantiates these observations with detailed roofline analyses.

The goal of this work is to break L2 isolation and achieve beyond-linear scaling. While distributed execution treats remote access as overhead to be minimized, we take the opposite view: remote access is not the problem but the opportunity. Our **key insight** is that *the remote accesses that distributed execution strives to eliminate, when architecturally managed through dedicated orchestration, are exactly the key to unlocking beyond-linear scaling*. When multiple SMs on different dies repeatedly access the same data through SM-direct remote loads, every access is served by the one L2 slice where that data reside, achieving cross-die L2 sharing. The effective L2 capacity for data reuse thus spans all participating dies, breaking L2 isolation and enabling beyond-linear scaling. This reframes remote loads from a communication overhead to be eliminated into the critical primitive to enable L2 sharing. Single-GPU execution also performs remote loads, but without architectural management the resulting traffic overwhelms the interconnect.

Architecturally managing SM-direct remote loads is non-trivial, with two critical challenges: ① Architecture-unaware task scheduling and data layout cause all-to-all remote traffic with high volume, high latency, and poor load balance. ② Existing GPU latency-hiding mechanisms cannot tolerate the long and diverse remote-access latencies at wafer scale, severely degrading compute utilization. To address these challenges, we propose HALO (**H**ardware-**A**ware **L**oad **O**rchestration), a hardware-software co-design framework that makes SM-direct remote loads efficient for practical L2 sharing at wafer scale. ❶ *Remote-access scheduling* at both operator and system levels reduces traffic volume, latency, and imbalance through topology-aware data placement, complemented by lightweight hardware support. ❷ *Speculative pipelining* defers shared-memory allocation to eliminate redundant occupation. This creates sufficient pipeline depth to hide long remote-access latency.

In summary, this paper makes the following contributions:

- We analyze existing wafer-scale GPU LLM training solutions, reveal the gap between their linear scaling and the ideal beyond-linear scaling, identifying L2 isolation as the root cause.
- We propose HALO, a hardware-software co-design framework for wafer-scale GPU LLM training that makes SM-direct remote loads efficient for L2 sharing. HALO introduces remote-access scheduling to reduce communication overhead, along with speculative pipelining to hide long-latency remote accesses.
- Evaluated on cycle-accurate simulator with diverse LLMs, HALO achieves 1.24× average speedup over state-of-the-art distributed execution, reaching 98% of theoretical beyond-linear bound.

2 Background and Motivation

2.1 Wafer-Scale GPUs

Enabled by advanced packaging technologies such as TSV [6, 14, 18], CoW [10, 11, 51], and RDL [8, 9, 19, 20], wafer-scale chips have emerged as a promising scale-up solution [21, 32, 40, 41, 46–49, 53]. Wafer-scale integration mainly follows two routes. Monolithic designs, e.g., Cerebras [21], fabricate compute and SRAM on one wafer-scale substrate but cannot place HBM stacks beside each die.

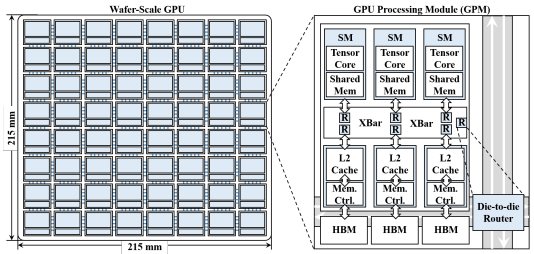


Figure 2: Organization of a wafer-scale GPU. 64 GPMs are interconnected in an 8×8 2D mesh on a single wafer substrate.

Chiplet-based integration [32, 37, 40] instead assembles known-good dies on a wafer-level substrate, improving yield and enabling co-packaged HBM. This route is practical: a wafer-level GPU prototype [32] gives each die its own HBM, while TSMC SoW-X [37] integrates ASICs with HBM on a system-on-wafer platform. Following prior architectural studies [32, 41, 46–49], we adopt an HBM-equipped chiplet organization. It improves flexibility and yield while providing the memory capacity required by LLMs, addressing capacity limitations of SRAM-only design. This is also consistent with TSMC SoW-X direction [37].

As shown in Figure 2, we model a wafer-scale GPU as an 8×8 2D mesh of 64 GPU processing modules (GPMs) on a 215 mm×215 mm wafer substrate. Advanced packaging provides 5 TB/s per-link die-to-die bandwidth, as achieved in CoWoS-based production multi-die GPUs [30, 43] and scaled to wafer scale like SoW-X [37], with ~100 ns latency [40]. This is a substantial improvement over conventional chip-to-chip interconnects [29, 31]. Each GPM contains one compute die and HBM stacks. The compute die uses a NoC to connect SMs, memory partitions with L2 caches and memory controllers, and die-to-die routers for cross-die memory accesses. This organization matches prior wafer-scale and multi-die GPU designs [1, 30, 32, 48]. It can be programmed either transparently as a single GPU or with distributed execution, where each die acts as an independent GPU and communication uses optimized communication operators as in multi-GPU systems.

2.2 Revisiting the Scaling of Wafer-Scale GPUs

2.2.1 Linear Scaling of Distributed Execution. The most natural approach is to program the entire wafer as a single GPU using the same interface as a conventional GPU, so that existing programs transparently scale without awareness of the multi-die structure. NVIDIA B200 adopts this approach at the two-die scale: the address space is interleaved across both dies, and kernels launch over the entire GPU without software awareness of where each thread block executes or where data reside [30]. With 10 TB/s NV-HBI, communication is not a bottleneck. At wafer scale, however, the dramatic increase in die count fundamentally changes the situation. Global all-to-all accesses now span tens of dies, causing die-to-die traffic to surge. On a mesh topology with limited bisection bandwidth, even the high-bandwidth wafer fabric cannot prevent communication from becoming the bottleneck. The larger die count also increases access latency substantially. Our experiments in Section 4.2.1 confirm that compute utilization falls below 30%.

Although a wafer-scale GPU can support shared address space and direct remote accesses of single-GPU execution, the prohibitive communication overhead forces LLM systems to retreat to

distributed execution in practice. The system is instead treated as a tightly integrated multi-GPU system, where communication-centric optimizations such as collective operators and compute-communication fusion substantially reduce overhead. On our wafer-scale GPU (configuration in Section 4.1), communication time is less than 10% of compute and is effectively hidden, enabling linear scaling: when resources, including total die area and HBM bandwidth, increase to $N\times$, performance also improves by $N\times$. In multi-GPU LLM literature, linear scaling is widely regarded as the optimum.

2.2.2 Beyond-Linear Scaling Potential. In fact, such linear scaling is only the optimum of treating additional dies as independent GPU instances, rather than the optimum of ideal single-GPU scale-up. Under distributed execution, each die performs majority of its computation locally, only inputs/outputs are exchanged with communication. Consequently, if a single die is configured at the *sweet spot*, a multi-die GPU under distributed execution remains at the same per-die sweet spot: each die still executes the same local computation independently. Therefore, distributed execution behaves as multiple independent instances, whose best-case performance scales linearly with the number of dies, as shown in Figure 1(a).

However, an ideal single GPU should exhibit *beyond-linear* scaling: increasing resources, including total die area and HBM bandwidth, to $N\times$ can yield more than $N\times$ performance, as illustrated in Figure 1(b). The roofline model in Figure 1(c) makes this point clear. Consider a single-die GPU with an SM-L2-DRAM configuration that lies at the sweet spot, namely at the intersection of the compute roof and the memory-bandwidth roof. When die area and DRAM bandwidth are doubled, if SM count and L2 capacity are also doubled proportionally, performance also doubles, and the balance point remains at the same operational intensity. However, the enlarged L2 shifts the effective operating point to the right, transforming the scaled GPU into a compute-bound design. In other words, *memory-system redundancy* emerges. To achieve ideal scaling, the GPU must rebalance compute and memory resources by reducing L2 cache capacity ("–L2 Cache") and reallocating the freed area to additional SMs ("+SM") so as to return to the sweet spot. Therefore, under ideal single-GPU scaling, doubling die area and DRAM bandwidth achieves >2× performance, i.e., ideal single-GPU scaling is beyond linear. This indicates a gap between the linear scaling of distributed execution and the ideal beyond-linear scaling.

2.2.3 L2 Isolation: The Root Cause. The root cause of this gap is *L2 isolation* induced by the communication-centric philosophy of distributed execution. Reducing communication is the primary objective, which fundamentally precludes exploiting remote L2 caches for data reuse: reusing a remote L2 requires repeatedly accessing the same remote data. Consider AllGather-GEMM as an example in Figure 3(a). With separate collective and compute operators, remote data are first copied into local DRAM and then repeatedly read, so each die exploits only its own L2 for data reuse. Some compute-communication fused kernels alter the dataflow so that matrix A is loaded only once into the SM and reused as each TB (Thread Block) iterates along the remaining dimension. However, this shifts the repeated access to matrix W and B, which still exploit only local L2 caches. This merely changes *which* data are repeatedly accessed without breaking the isolation itself. Under L2 isolation, when SM count, L2 capacity, and DRAM bandwidth all scale by N,

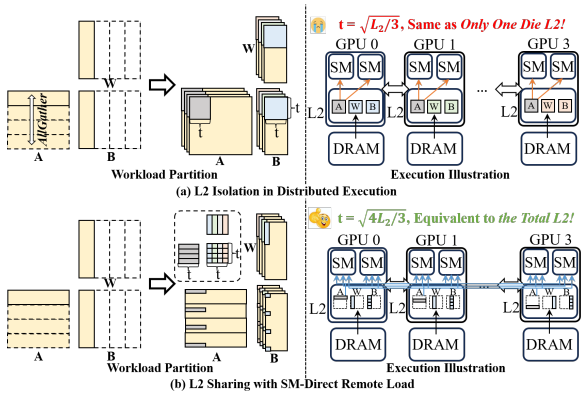


Figure 3: L2 isolation vs. L2 sharing, illustrated with a GEMM example. (a) Under distributed execution, each die exploits only its local L2 for data reuse, confining the effective L2 capacity to a single die. (b) With SM-direct remote loads, SMs load directly from remote L2-DRAM, extending effective L2 capacity across all dies. Local W and B accesses are omitted.

arithmetic intensity cannot rise. The scaled GPU remains pinned at the balance point and cannot release L2 area to SMs. Beyond-linear scaling is therefore unachievable under distributed execution.

2.3 Design Philosophy and Challenges

The above analysis identifies L2 isolation as the architectural root cause of the gap to beyond-linear scaling. On conventional multi-GPU systems, this isolation is inescapable: their limited inter-chip bandwidth cannot sustain the repeated remote accesses that L2 sharing demands. Wafer-scale GPUs, with their substantially richer inter-die fabric, create a new opportunity. Our design philosophy is to *embrace remote access rather than eliminate it, and architecturally manage the resulting traffic through dedicated architectural orchestration to unlock beyond-linear scaling*. As shown in Figure 3(b), when multiple SMs on different dies repeatedly access the same data through SM-direct remote loads, every access is served by the one L2 slice where that data reside, achieving cross-die L2 sharing. Effective L2 capacity for data reuse thus spans all participating dies, breaking L2 isolation and enabling beyond-linear scaling. Single-GPU execution also performs remote loads, but without architectural management the resulting traffic overwhelms interconnect.

Making SM-direct remote loads efficient for practical L2 sharing is non-trivial. Two challenges must be addressed to achieve architecturally managed remote access:

C1: Architecture-unaware scheduling introduces excessive communication overhead. Without topology and workload awareness, scheduling and data layout create high-volume, high-latency, poorly balanced global all-to-all traffic. For example, B200-style address interleaving [30] spreads accesses across all dies, causing long-distance traffic that occupies many links, making bandwidth the bottleneck and incurring high latency. It also fails to align with hardware execution patterns, leading to imbalanced data transfer and memory accesses. HALO addresses this with operator- and system-level *remote-access scheduling* that aligns data placement and scheduling with compute and topology.

C2: Existing latency-hiding mechanisms cannot tolerate long remote-access latency. SM-direct remote loads across an

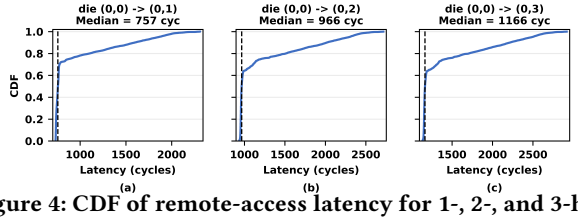


Figure 4: CDF of remote-access latency for 1-, 2-, and 3-hop transactions on the wafer-scale GPU.

L2-sharing domain of dimension d traverse $2(d-1)$ die-to-die links at ~100 ns each. For d=4, this adds 600 ns beyond the ~500 ns local round trip. Figure 4 shows median remote latency of 757–1166 cycles for 1–3 hops, with 2500–3000-cycle tails from queuing. Although GPUs hide memory latency via multi-stage software pipelining, each stage reserves a shared-memory tile, limiting depth. Our experiments show that it requires at least 12 stages for >98% compute utilization, but H100 shared memory supports only 7 stages, yielding merely 68.5% utilization. Halo addresses this with *speculative pipelining*, which defers shared-memory allocation to provide enough pipeline depth for long remote latencies.

3 Halo Design

3.1 Overview

As established in Section 2, architecturally managed remote access is the key to making SM-direct remote loads efficient for practical L2 sharing at wafer scale. Halo addresses the two challenges identified in Section 2 through dedicated hardware-software co-design:

- To address **C1** (communication overhead), Halo introduces *remote-access scheduling*, detailed in Section 3.2, which manages remote-access traffic through compute-aligned interleaving at operator level and domain-partitioned execution at system level.
- To address **C2** (latency hiding), Halo introduces *speculative pipelining*, detailed in Section 3.3, an architectural design that employs on-demand shared-memory allocation to create sufficient pipeline depth for hiding long remote-access latencies.

3.2 Remote-Access Scheduling

Remote-access scheduling addresses C1 by managing SM-direct remote loads through data placement and system-level partitioning.

3.2.1 Key Insight: Compute-Topology Co-Alignment. Reducing communication overhead of SM-direct remote loads requires jointly optimizing traffic volume, latency, and load balance. Global spreading causes excessive traffic volume and long latency, while simple partitioning reduces them but creates imbalance. The key observation is that operators in LLM training exhibit regular, tile-based access patterns. It allows communication traffic to be *managed* by aligning data placement with both compute pattern and mesh topology, achieving low traffic volume, short latency, and balanced load simultaneously. Remote-access scheduling exploits this at two levels: *compute-aligned interleaving* confines and balances tile-load traffic at operator level, while *domain-partitioned execution* limits communication scope by partitioning wafer into L2-sharing domains.

3.2.2 Compute-Aligned Interleaving. Using GEMM as the representative operator, we characterize the communication cost of SM-direct remote loads along three dimensions:

- **Maximum link load** $load_{max}$: the maximum amount of data transmitted on any single link, with the total memory-access volume of one die normalized to 1. This captures *traffic volume*.
- **Maximum hop count** hop_{max}: the longest path traversed by any remote memory access. This captures *latency*.
- **Access balance**: uniformity of memory accesses from TBs with the same index on each die at the same *K-step*. A *K-step* denotes an iteration over the tiled GEMM reduction (K) dimension, where a TB loads the corresponding A and W tiles and accumulates their product into its output tile B. This captures *load imbalance*.

These objectives appear to conflict: global interleaving achieves balance but maximizes traffic volume (e.g., $load_{max}$) and latency, while partition reduces traffic volume and latency but introduces imbalance. We show that all three can be simultaneously achieved by *aligning fine-grained interleaving with compute access pattern within a topology-confined region*. We arrive at this design through a systematic exploration of data-layout design space shown in Figure 5.

Global Interleaving. Global sub-page interleaving, as in NVIDIA B200 [30], implicitly balances load but spreads all traffic across all dies, as shown in Figure 5(a). For a $p \times p$ mesh with uniform all-to-all accesses, $load_{max}$ is $p/4$, and hop_{max} is $2(p-1)$, corresponding to diagonal accesses. $p/4$ comes from the central bisection. Each source-destination pair carries normalized traffic $1/p^2$. Across the central bisection, $p^2/2$ source dies communicate with $p^2/2$ destination dies on the other side, giving one-direction bisection traffic $(p^2/2)(p^2/2)(1/p^2) = p^2/4$. This traffic is shared by p links across the bisection, yielding $load_{max} = p/4$. Because of globally interleaved address, accesses are implicitly balanced [25]. However, severe traffic volume ($load_{max} = p/4$) and high latency ($hop_{max} = 2(p-1)$) make this impractical at wafer scale.

1-D Partition. To reduce communication cost, we first consider a partition inspired by tensor parallelism: 1-D partition localizes one input operand and output, so only *half* of memory accesses are remote, halving $load_{max}$ to $p/8$ as shown in Figure 5(b). However, hop_{max} remains $2(p-1)$ because remote accesses still span all dies, so long latency persists. It also creates imbalance: at *K-step*=1 for the first TB, all dies access the same first-row tile of matrix A, concentrating all remote accesses on die $(0, 0)$.

2-D Partition. The failure of 1-D partition to reduce latency stems from its inability to avoid long-distance accesses such as diagonal ones. To address this, we narrow the access-spreading region by partitioning both operands in two dimensions, as in Figure 5(c). Each die accesses only dies in the same row and column, reducing hop_{max} to $p-1$. This narrowing also reduces the number of traversed links, maintaining $load_{max}$ at $p/8$. Unlike prior 2-D distributed execution, which localizes computation to avoid remote access, our design uses SM-direct remote loads to enable cross-die L2 sharing. However, this coarse-grained partition still suffers from imbalance: at *K-step*=1, accesses to matrix A all fall on the first-column dies, while accesses to W all fall on the first-row dies.

Compute-Aligned Interleaving. The analysis above reveals that coarse-grained partitioning inherently causes imbalance: achieving balance requires distributing accesses within each tile load evenly, as global interleaving does, but within a confined region. Our compute-aligned interleaving achieves precisely this, as shown in Figure 5(d). Unlike coarse-grained 2-D partitioning, we interleave

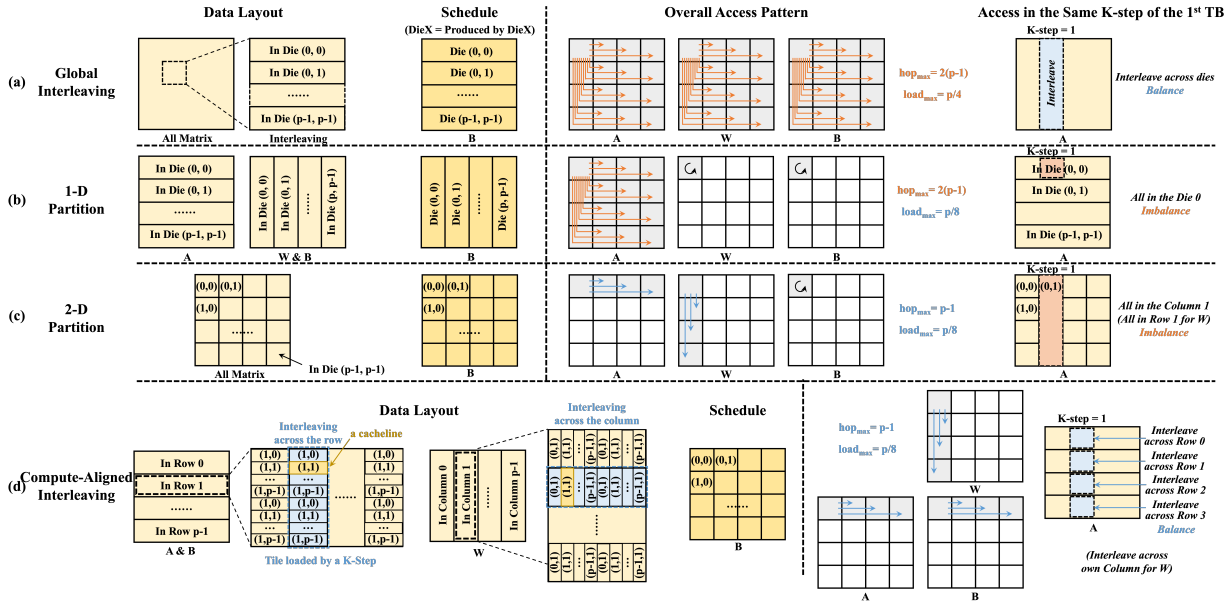


Figure 5: Derivation of compute-aligned interleaving, evaluated by maximum link load ($load_{max}$), maximum hop count (hop_{max}), and access balance. (a) Global interleaving spreads all accesses across all dies: balanced but with excessive traffic volume ($load_{max}$=$p/4$) and latency (hop_{max}=$2(p-1)$). (b) 1-D partition halves traffic volume but retains full latency and introduces severe imbalance. (c) 2-D partition reduces latency to $p-1$ but still suffers from imbalance. (d) Compute-aligned interleaving achieves low traffic volume, short latency, and balance simultaneously by fine-grained interleaving within a topology-confined region.

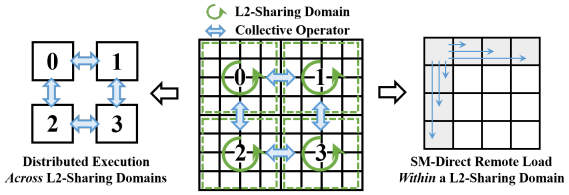


Figure 6: Domain-partitioned execution partitions the wafer into L2-sharing domains (four 4×4 domains shown). Within each domain, SM-direct remote loads under compute-aligned interleaving enable cross-die L2 sharing. Across domains, distributed execution handles data exchange via collectives.

rows $[(M/p)i, (M/p)(i{+}1))$ of matrix A across the dies in the i-th die row at row granularity, so that all memory accesses corresponding to a tile load are evenly distributed across all dies in that row. Similarly, matrix W is interleaved across a die column at column granularity. The output matrix B adopts the same layout as A to ensure the subsequent GEMM receives correctly laid-out input. The layout of B does not materially affect communication cost, because it is written only once and this cost is negligible compared with the repeated accesses to A and W. Under this layout, $load_{max}$ and hop_{max} remain halved as in 2-D partition, while the fine-grained, compute-aligned interleaving ensures that each tile load is evenly distributed across the dies in a row or column, achieving load balance.

For FlashAttention, Q, K, V, and O all follow this layout. Within each L2-sharing domain, computation is scheduled head by head to maximize L2 sharing, since different heads share no data. Non-GEMM operators such as LayerNorm and Softmax have little data reuse, so each die processes its local partition with local accesses. Their small runtime does not reduce the overall benefit. In the backward, GEMM operand roles are transposed, so we insert a lightweight layout transformation between forward and backward. It moves each element once, takes less than 2% of total time, and is outweighed by the benefit of compute-aligned interleaving.

3.2.3 Domain-Partitioned Execution. Compute-aligned interleaving optimizes the communication pattern at the operator level, but when the L2-sharing domain spans the entire wafer, the absolute communication overhead remains non-trivial due to the large number of dies. To further reduce this overhead, we introduce domain-partitioned execution, a system-level scheduling strategy that limits the scope of L2 sharing to a subregion of the wafer.

Execution Model. As shown in Figure 6, domain-partitioned execution divides the wafer into L2-sharing domains. Within each domain, SM-direct remote loads with compute-aligned interleaving enable cross-die L2 sharing. Across domains, distributed execution performs only communication-light data exchanges. This reduces $load_{max}$ and hop_{max} from $p/8$ and $p-1$ to $d/8$ and $d-1$, where d is the domain dimension, trading a modest L2-sharing scope reduction for much lower communication overhead.

Data Exchange Traffic. Cross-domain transfers occur only between dies with the same in-domain position. In Figure 6, die $(0,0)$ exchanges A only with die $(0,4)$ and W only with die $(4,0)$. The exchanged data preserves compute-aligned interleaving, so intra-domain execution is unchanged. Computation within a domain and communication across domains are overlapped: as soon as a remote chunk arrives, its computation starts, pipelining inter-domain transfer with intra-domain execution.

3.2.4 Architectural Support. Remote-access scheduling requires two hardware extensions, shown in Figure 8: a TMA extension for fine-grained interleaved layouts and traffic-class isolation for intra-/inter-domain traffic.

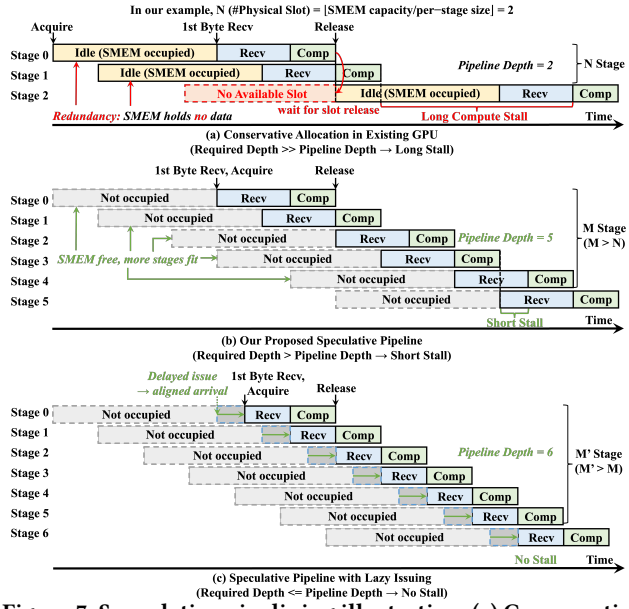


Figure 7: Speculative pipelining illustration. (a) Conservative allocation limits pipeline depth to N, causing compute stalls. (b) On-demand allocation enables $M{>}N$ stages but a residual stall remains from arrival-time spread. (c) Lazy-issuing compresses the spread, eliminating stall.

TMA Extension. Existing TMA assumes standard matrices in contiguous address spaces, while compute-aligned interleaving distributes each matrix across dies. We extend tensor descriptor of TMA with `base_per_die[d]`, the base address on each die, and `interleave_dim`, the interleaving dimension. These fields initialize a *die base address table* at kernel launch. The TMA address generator is also extended to resolve each element to its die and local address. For row-wise interleaving: For an element at position (row, col) within the loaded tile under row-wise interleaving:

$$\texttt{die_id} = \texttt{row} \bmod p,$$

$$\texttt{offset} = \lfloor \texttt{row}/p \rfloor \times \texttt{stride} + \texttt{col},$$

$$\texttt{addr} = \texttt{base_per_die[die_id]} + \texttt{offset},$$

where p is the number of dies along the interleaving dimension. Column-wise interleaving swaps row and column. Each request carries `addr` and `die_id` for routing. The extension is confined to TMA address generator without modifying the memory hierarchy.

Traffic-Class Isolation. To reduce interference between intra- and inter-domain traffic, each request carries *D-Flag* as in Figure 8. At each D2D router output port, the flag selects either *intra-queue* for L2-sharing traffic or *inter-queue* for cross-domain exchange. A weighted arbiter allocates bandwidth between them using a weight derived from traffic volumes and configured at kernel launch.

3.3 Speculative Pipelining

Even with remote-access scheduling, remote latency still remains much higher than local latency, as in Section 2. If exposed, it creates pipeline bubbles and lowers compute utilization. GPUs hide latency with multi-stage software pipelining, but existing shared memory management limits pipeline depth, which is insufficient to hide latency. Speculative pipelining addresses C2 by *speculatively* issuing more tile-load requests than physical shared memory can simultaneously hold, eliminating redundant occupation and creating enough depth to hide long remote latency.

3.3.1 Key Insight: On-Demand Allocation. The root cause of insufficient pipeline depth is the overly conservative shared-memory management in existing GPUs. As Figure 7(a) shows, the pipeline depth equired to hide latency is $\lceil \text{latency}/\text{Comp} \rceil + 1$, which rises sharply with remote latency. In existing implementations, shared memory for a given stage must be acquired *before* the tile-load instruction is issued, but this space stays idle, holding no valid data, until the first byte returns to the SM. This issue-to-first-byte occupation is *entirely redundant* and limits effective depth under fixed capacity.

On-demand allocation eliminates this redundancy by deferring the acquisition until the first byte of a tile load arrives, as in Figure 7(b). Thus, outstanding tile loads can exceed physical shared-memory capacity, enabling more stages. The approach is *speculative*: it bets earlier stages will free slots before later responses arrive. A large arrival-time spread still causes stalls, lazy-issuing in Section 3.3.3 mitigates this. Overflow handling in Section 3.3.2 preserves correctness when slots are temporarily exhausted.

3.3.2 Virtual Pipeline Stages. On-demand allocation requires the hardware to dynamically map tile-load responses to physical shared-memory slots at response arrival time. This timing is not deterministic and cannot be managed by software. We therefore introduce *virtual pipeline stages*, a hardware-managed abstraction that allows the software pipeline depth to exceed the physical shared-memory capacity. Virtual pipeline stages provide a virtual shared-memory space and virtual barriers, allowing the programmer to write multi-stage pipelines exactly as before, while the hardware transparently performs on-demand physical resource allocation.

Programming Model. The programmer specifies M virtual pipeline stages ($M \gg N$, where $N = \lfloor \text{SMEM capacity}/\text{per-stage size} \rfloor$), allocates virtual shared-memory buffers and virtual barriers for all M stages, and writes the pipeline loop identically to the standard case, only replacing physical shared-memory and mbarrier references with their virtual counterparts. The hardware transparently manages the mapping from virtual to physical resources on demand.

Allocation Protocol. Physical shared memory is partitioned into N fixed-size slots, each paired with a physical mbarrier. A VSHMEM table in SM manages the dynamic mapping from M virtual stages to the N physical slots. When a kernel acquires a virtual barrier, no physical resource is allocated. The VSHMEM table records the pending acquisition and the mapping from outstanding TMA requests to the virtual-stage index. When the *first response* for a virtual stage arrives at the SM, a free physical slot and its paired mbarrier are allocated, the virtual-to-physical mapping is recorded, and the response data is written to the physical slot. Subsequent responses for the same stage are routed via recorded mapping. Once all responses arrive, the physical mbarrier completes and the virtual barrier's phase bit is updated. The consumer waits on the virtual barrier, accesses data in the physical slot, and upon completion releases the virtual barrier, freeing both physical slot and mbarrier for reuse.

Overflow Recovery. Since virtual stages outnumber physical slots, a response may arrive when all slots are occupied. VSHMEM marks its stage and retries this transaction. When a slot is freed, it is assigned to this stage. There is no deadlock because each occupied

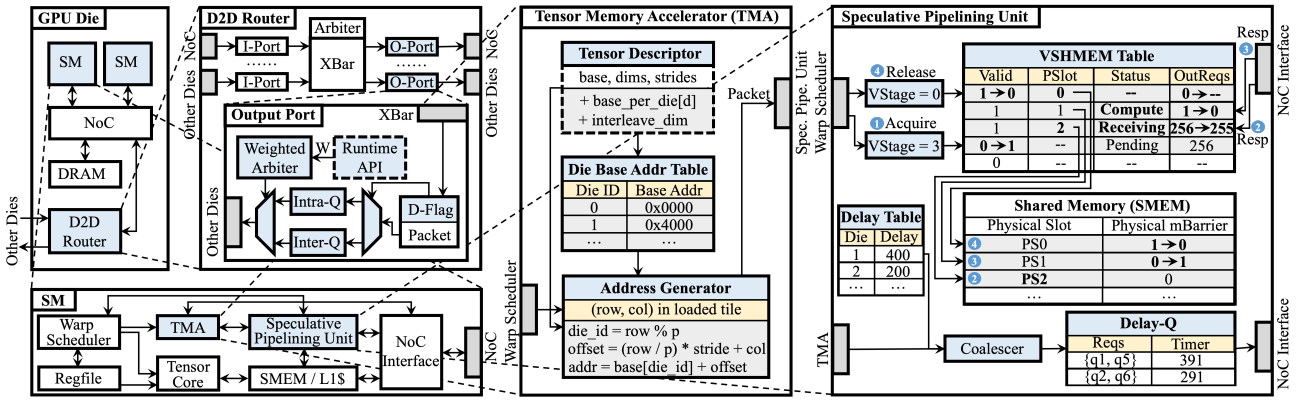


Figure 8: Architectural design of Halo, showing three hardware extensions integrated into the GPU die. The TMA is extended with a die base address table and an interleaving-aware address generator. The D2D router output port is augmented with traffic-class isolation using a D-Flag and weighted arbiter. A speculative pipelining unit manages virtual-to-physical stage mapping via the VSHMEM table and aligns request arrival times through lazy-issuing hardware.

slot eventually receives all responses through deadlock-free mesh and releases its slot after computation, ensuring forward progress.

Warm-up. Halo throttles the pipeline depth to avoid transient overflow during warming up. A TB starts with the active pipeline depth equal to the number of physical slots, as in a conventional pipeline. Each time a stage releases a physical slot, the active pipeline depth is increased by one until reaching virtual pipeline stage M. This prevents the initial long-latency stages from being launched simultaneously before any physical slot can turn over.

3.3.3 Lazy-Issuing. To allow rapid turnover of physical slots, slot occupation time must be minimized. Occupation spans from the first-byte arrival to the consumer completion. Arrival-time spreading dominates the time: data is interleaved across dies at different distances, so individual transactions of a tile load have vastly different latencies, prolonging the interval from first- to last-byte arrival.

Lazy-issuing reduces this spread by deliberately delaying the issuance of short-latency requests so that all requests complete at approximately the same time. Each request is delayed by the difference between the static latency to the farthest die and the computed static latency of that particular request, eliminating the arrival-time spread from static-latency differences. To further align the spread caused by queuing delay, each die maintains counters that track the observed access latency to all other dies. Once latency reaches a steady state, the per-request delay also accounts for the measured queuing-latency difference, further tightening alignment. By shortening per-stage occupation time, lazy-issuing enables fewer physical slots to sustain more virtual pipeline stages, eliminating the residual stall as shown in Figure 7(c).

3.3.4 Architectural Support. Speculative pipelining requires three architectural supports: ISA extensions to expose virtual pipeline stages, a VSHMEM table to manage the virtual-to-physical mapping, and lazy-issuing logic to align request arrival times.

ISA Extension. Four PTX instructions that constitute TMA-based pipelining require extension to operate on virtual addresses: `cp.async.bulk.tensor` (data transfer), `mbarrier.arrive.expect_tx` (producer acquire), `mbarrier.try_wait.parity` (producer/consumer wait), and `mbarrier.arrive` (consumer release). We add a virtual state-space qualifier, analogous to the existing `shared::cta` and `shared::cluster` qualifiers in PTX. When operating on virtual addresses, these instructions interact with the VSHMEM table rather than directly accessing physical shared memory and mbarriers.

VSHMEM Table. The VSHMEM table maintains one entry per virtual pipeline stage. Each entry stores four fields: a `Valid` bit, a physical-slot index `PSlot`, a `Status` word, and an outstanding-response counter `OutReqs`. As illustrated in Figure 8, each virtual stage follows a four-step lifecycle: ① **Acquire.** The warp scheduler acquires a virtual barrier for stage i. The VSHMEM table sets `Valid`←1, `PSlot`←unassigned, `Status`←Pending, and `OutReqs`←the total expected response count. No physical shared memory is consumed at this point. ② **Allocate & Receive.** When the first network response for stage i arrives, the table allocates a free physical slot and its paired physical mbarrier, records the mapping in `PSlot`, and transitions `Status` to Receiving. Each subsequent response is routed to the mapped physical slot via an associative lookup on the virtual-stage index, and `OutReqs` is decremented. ③ **Compute.** When `OutReqs` reaches zero, all tile data have arrived. The physical mbarrier completes and signals the consumer warp. `Status` transitions to Compute, and the consumer reads data directly from the physical slot. ④ **Release.** The consumer finishes and releases the virtual barrier. The table sets `Valid`←0, frees the physical slot and its mbarrier, making them available for reuse by subsequent stages. With M=12 virtual stages, the table contains only 12 entries. The associative lookup on each returning response completes in a single cycle, lies entirely off the SM's compute critical path, and is triggered asynchronously by network responses.

Overflow Handling Workflow. Overflow recovery is in cache-line granularity. The workflow is as follows: 1) When a response arrives, hardware checks its virtual-stage entry in VSHMEM table. If the entry is Pending and no physical slot is free, the response overflows, the entry stays unmapped and is marked Critical. 2) The overflow immediately triggers a retry of its request, injected into network with a high priority. 3) Once a slot is released, VSHMEM assigns it to the critical virtual stage and moves it to Receiving. 4) The retried response is accepted in Receiving; if the entry is still Pending, it triggers another retry.

Lazy-Issuing Hardware. As shown in Figure 8, the lazy-issuing mechanism consists of delay table and delay queue. A *delay table*

Wafer-Scale GPU		Per-Die Configuration		
Parameter		Parameter	Sweet-spot	Halo
Dies	64	SMs per die	64	80
Topology	8×8 mesh	FP16 (TFLOPS)	959	1199
D2D bandwidth	5 TB/s	L2 cache	50 MB	4 MB
D2D latency	100 ns	HBM bandwidth	512 GB/s	

Table 1: Hardware configuration. Both Sweet-spot die and Halo die occupy the same die area.

records the calculated delay to each destination die, which is calculated as the difference between the maximum latency across all destinations and the latency to the specific target die, as measured by dedicated latency counters. For a 4×4 domain this amounts to 16 entries of 2 bytes each. During warming up, the delay table is updated from the latency counters whenever a stage releases a physical slot. *Delay queue* buffers delayed requests after coalescing. The coalescer merges requests from the same TMA instruction and destination die when their addresses follow a stride pattern. Each entry contains a 74-bit request descriptor: `Valid` (1b), `TMAId` (5b), `DstDie` (6b), `BaseAddr` (48b), `Stride` (8b), and `Count` (6b), plus a 12-bit `Timer`. When the timer expires, the queue regenerates requests from `BaseAddr`, `Stride`, and `Count`. In our kernels, each 128-request TMA targets four destinations, thus coalescing into four entries. With M=12 virtual stages and two TMA loads per stage, the bound is 12×2×4 = 96 entries per SM, totaling 96×86 bits ≈1KB. All structures operate asynchronously relative to the SM's warp-scheduler critical path and do not affect SM frequency or pipeline timing.

4 Evaluation

4.1 Methodology

4.1.1 Simulation Infrastructure. We evaluate Halo using a cycle-accurate wafer-scale GPU simulator that integrates a customized Accel-Sim [17] with BookSim2 [13]. We extend Accel-Sim's Hopper model to better capture TMA and multi-stage software pipelining with asynchronous barriers. We also parallelize per-die simulation and synchronize at cycle boundaries for acceleration. Validation against H100 shows GEMM errors within 6% across diverse shapes.

4.1.2 Hardware Configuration. Table 1 lists hardware parameters. Our wafer-scale GPU has 64 dies in 8×8 2D mesh, with 512 GB/s HBM per die. D2D link has 5 TB/s bandwidth with 100 ns latency.

Sweet-spot die. Due to area constraints at wafer scale, each GPU die cannot replicate the full H100 design. We therefore adopt a training-optimized die that doubles the tensor-core count per SM while reducing the total SM count to 64, achieving comparable peak throughput (959 vs. 989 TFLOPS in FP16) in a smaller area. This design trend of maximizing datapath density while reducing per-SM control overhead aligns with the industry-wide shift toward larger execution units in training-optimized accelerators. The L2 cache is set to 50 MB, occupying approximately 23% of total die area, preserving the original compute-to-L2 ratio of H100. This die operates near the *sweet spot* of roofline model. Distributed execution baselines all use wafer-scale GPUs built from this sweet-spot die.

Halo die. To realize beyond-linear scaling, Halo reduces per-die L2 from 50 MB to 4 MB. Released area, approximately 18% of total die area, is conservatively reallocated to additional SMs, yielding 80 SMs per die, a 25% increase over sweet-spot die. With the same per-SM throughput, the theoretical performance upper bound of Halo is 80 / 64 = 1.25× over the sweet-spot baseline. Both die configurations occupy the same total die area, ensuring an iso-area comparison. Although each die retains only 4 MB of local L2, L2 sharing across a 4×4 domain provides an effective L2 capacity of 4 MB×16 = 64 MB for data reuse, comparable to the 1.25× 50 MB on the sweet-spot die. Halo and single-GPU execution baselines adopts this die.

Name	Hidden Size	FFN Hidden Size	Attention Heads	Sequence Length	Batch Size
Llama	16384	53248	128	2048	8
GPT3	12288	49152	96	2048	8
Llama-L	65536	212992	256	4096	16
GPT3-L	49152	196608	192	4096	16

Table 2: LLM workload configurations.

Area model. Our area estimates are grounded in publicly available die analyses of NVIDIA H100 [29], where the L2 cache subsystem (SRAM arrays, tag logic, and controllers) occupies approximately 20% of total die area. We adopt a conservative reallocation ratio: of the 46 MB of freed L2 area, only the SRAM array portion (which dominates L2 area) is assumed usable for SM placement. The remaining tag logic, controllers, and associated routing and power-delivery infrastructure are excluded. We further account for physical design constraints: SM macros have a coarser placement granularity than SRAM arrays, and the freed region must accommodate the SM's local interconnect and register-file area without violating metal-layer routing density limits. These constraints mean that not all of the 18.4% freed area translates directly into SM count, making the resulting 16 additional SMs a conservative estimate. To quantify sensitivity, we note that if the reclaimable area were 15% instead of 18.4%, the SM count would increase to only 74 (a 1.16× bound), while 21% reclaimable area would yield 84 SMs (a 1.31× bound). Beyond-linear scaling opportunity exists across this entire range.

4.1.3 Workloads. We evaluate Halo on four LLM training workloads from two model families: Llama 3.1 405B [5] (Llama) and GPT-3 175B [2] (GPT3). To assess scalability to larger models, we derive scaled-up variants, Llama 3.1 Large (Llama-L) and GPT-3 Large (GPT3-L). Table 2 lists detailed configurations. GEMM kernels use a CUTLASS-like implementation exploiting TMA and WGMMA instructions with warp specialization for multi-stage pipelining.

4.1.4 Baselines. We compare Halo against six baselines spanning three execution paradigms:

- *Multi-GPU*: **TP-64G** [29] applies tensor parallelism across 64 GPUs, each comprising a single sweet-spot die.
- *Single-GPU Execution* on wafer-scale GPU: **Global Interleaving** [30] is the address-interleaving scheme adopted by NVIDIA B200 for transparent dual-die execution, extended to wafer scale. **LADM** [16] is a locality-centric data and threadblock management scheme for multi-die single-GPU execution.
- *Distributed Execution* on wafer-scale GPU: **2D-TP** is a widely adopted approach in WSE [7, 21], here we apply it to wafer-scale GPU. **TEMP** [47] proposes a novel TP and **WATOS** [46] proposes systematic coordination for LLM training on wafer-scale chips.

4.2 Overall Performance

4.2.1 End-to-End LLM Training. To evaluate the overall effectiveness of Halo, we compare end-to-end LLM training performance across all baselines and four workloads. Figure 9 presents the results

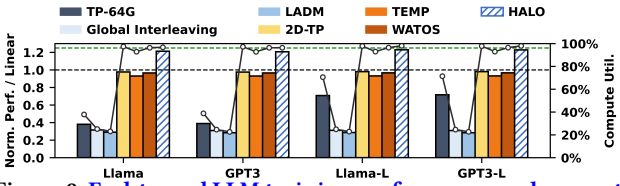


Figure 9: End-to-end LLM training performance and compute utilization, normalized to ideal linear scaling of 64 sweet-spot dies. HALO exceeds linear scaling on all workloads, approaching the theoretical beyond-linear bound.

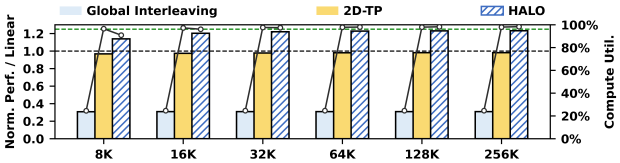


Figure 10: GEMM performance across matrix dimensions from 8K to 256K (M=N=K). HALO consistently outperforms both Global Interleaving and 2D-TP.

normalized to ideal linear scaling, i.e., 64× performance of a single sweet-spot die. The right y-axis also shows compute utilization.

HALO achieves 1.21–1.23× performance over ideal linear scaling across all workloads, reaching 96.4–98.1% of the theoretical beyond-linear scaling bound. Compute utilization consistently exceeds 95%, confirming that remote-access scheduling and speculative pipelining effectively manage communication overhead and keep the 80 SMs on the HALO die productively busy.

HALO on a single wafer outperforms TP-64G, a multi-GPU baseline with 64 GPUs, demonstrating the inherent advantage of wafer-scale integration. Compared to the single-GPU execution baselines, HALO achieves 3.92× and 4.27× average speedup over Global Interleaving and LADM. Compute utilization of Global Interleaving and LADM falls below 30%, as unmanaged all-to-all traffic severely overwhelms the interconnect, and LADM's locality optimization is even harmful due to the introduced communication imbalance.

Compared to distributed execution baselines, HALO achieves 1.24×, 1.31×, and 1.26× average speedup over 2D-TP, TEMP, and WATOS respectively. Although these baselines achieve near-linear scaling with only 2.2–6.9% performance loss through communication-centric optimization, all of them are fundamentally bounded by linear scaling due to L2 isolation, as analyzed in Section 2. HALO surpasses this barrier through cross-die L2 sharing. These results validate that HALO's L2-sharing approach achieves practical beyond-linear scaling on real LLM workloads.

4.2.2 GEMM Performance. To evaluate operator-level performance of HALO, we sweep square GEMM sizes from 8K to 256K (M=N=K) and compare HALO with Global Interleaving and 2D-TP. As Figure 10 shows, HALO consistently achieves 3.71–3.99× and 1.18–1.26× speedups over the two baselines respectively. Speedup grows with GEMM size as deeper-pipeline startup overhead is amortized. These results show that HALO's benefit is robust across operator sizes, not tied to specific workload configurations.

4.2.3 Scalability Evaluation. To validate the beyond-linear performance scaling of HALO, we evaluate different numbers of dies, with the die count corresponding to the resource scaling factor N.

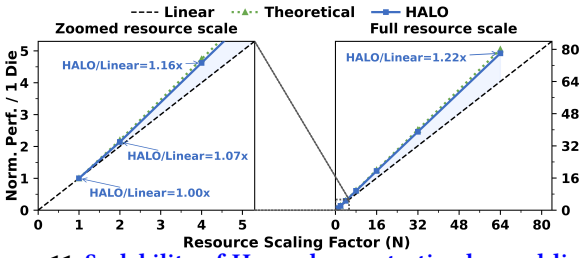


Figure 11: Scalability of HALO, demonstrating beyond-linear performance scaling and near-theoretical performance.

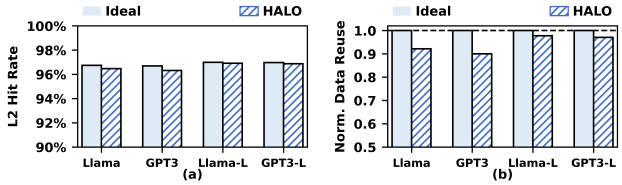


Figure 12: L2 sharing effectiveness compared with an ideal monolithic L2 of equivalent size. (a) L2 hit rate. (b) Data reuse.

We consider N (mesh shape) = 1 (1*1), 2 (1*2), 4 (2*2), 8 (2*4), 16 (4*4), 32 (4*8), and 64 (8*8), and perform a design-space exploration to identify the best SM-L2 configuration and L2-sharing domain size for each N. Figure 11 presents the results, with each point averaged across the four LLM workloads. "Linear" denotes linearly scaled performance relative to the single-die performance, whereas "Theoretical" denotes the theoretical performance under full utilization of each design. The results demonstrate that HALO achieves beyond-linear scalability with near-theoretical performance.

4.2.4 L2 Sharing Effectiveness. HALO's premise is that distributed L2 slices, accessed via architecturally managed remote loads, can act as a shared L2 with aggregated capacity. We validate this by comparing HALO with an ideal monolithic 64 MB L2 (4 MB×16). Figure 12(a) shows that HALO's average L2 hit rate, 96.7%, nearly matches the ideal 96.9%. Figure 12(b) shows normalized L2 reuse of 0.90–0.98 across workloads. These results confirm that cross-die L2 sharing under HALO effectively approximates an ideal monolithic cache and provides sufficient effective L2 capacity for beyond-linear scaling.

4.3 Ablation Study

To quantify individual contribution of each HALO component, we perform a cumulative ablation study starting from Global Interleaving and incrementally adding each technique, as shown in Figure 13.

Starting from Global Interleaving with a low compute utilization of 24.8% on average due to unmanaged all-to-all communication overhead, we incrementally add HALO's techniques. **+ Compute-Aligned Interleaving** (HALO-Layout) raises performance by 1.97× by confining remote accesses within the same row and column with fine-grained interleaving, substantially reducing traffic volume and latency. **+ Domain-Partitioned Execution** (HALO-Sched.) further improves performance by 1.37× by partitioning the wafer into L2-sharing domains, further reducing the remaining communication overhead and bringing utilization to 66.9%. At this point, the remaining performance gap is due to the elevated remote-access latency causing pipeline bubbles. **+ Speculative Pipelining** (HALO) delivers the final 1.45× boost by eliminating these pipeline bubbles,

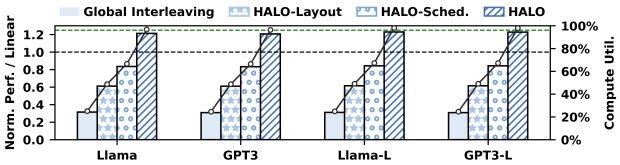


Figure 13: Cumulative ablation study. Starting from Global Interleaving, each HALO component is incrementally added: compute-aligned interleaving, domain-partitioned execution, and speculative pipelining.

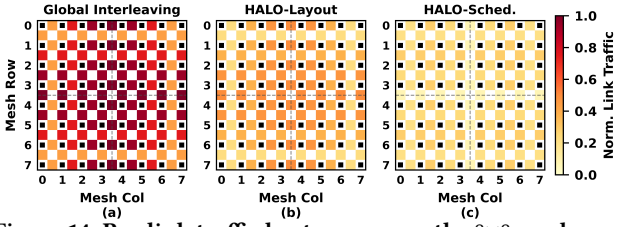


Figure 14: Per-link traffic heatmap across the 8×8 mesh, comparing three scheduling strategies. Compute-aligned interleaving halves the maximum link load relative to global interleaving, and domain-partitioned execution (d=4) further confines traffic to within each domain.

pushing utilization above 97.3% and enabling HALO to fully exploit the additional SMs on its 80-SM die. This ablation demonstrates that all three components are essential: remote-access scheduling alone is insufficient without latency hiding, and speculative pipelining alone cannot compensate for excessive communication overhead.

4.4 Remote-Access Scheduling Analysis

4.4.1 Traffic Volume Reduction. To validate that compute-aligned interleaving and domain-partitioned execution reduce traffic as designed in Section 3.2, we visualize the per-link traffic distribution across the 8×8 mesh under different scheduling strategies. Figure 14 visualizes the results. The maximum link load directly limits system performance, as it determines the bandwidth bottleneck.

With compute-aligned interleaving, the maximum link load is reduced by half compared to global interleaving, consistent with the theoretical analysis ($load_{max}$ from $p/4$ to $p/8$). Upon adding domain-partitioned execution with d=4, traffic is further reduced by narrowing the L2-sharing domain, reducing $load_{max}$ from $p/8$ to $d/8$. These results confirm that remote-access scheduling effectively controls traffic volume as predicted by the analytical model, and that the combination of operator-level and system-level optimization is necessary to bring communication overhead to a manageable level.

4.4.2 Domain Size Sensitivity. The L2-sharing domain size d governs the trade-off between L2-sharing scope and communication overhead: a larger domain provides more effective L2 capacity but incurs higher traffic, while a smaller domain limits communication but requires retaining more local L2, leaving less area for SMs. To identify the optimal operating point, we sweep d and re-balance the SM-L2 configuration for each size. Figure 15 presents the results.

Smaller domains (d=2) cannot achieve optimal performance because the large per-die L2 requirement occupies die area that could otherwise be allocated to SMs. Larger domains (d=8, full wafer) also underperform due to increased communication overhead that

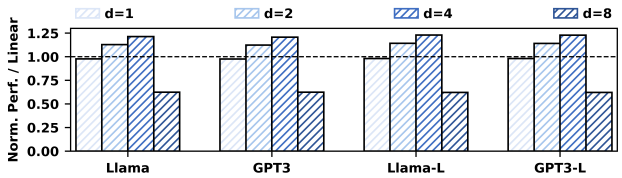


Figure 15: Sensitivity to L2-sharing domain size d on a fixed 64-die wafer. d=4 optimally balances L2-sharing scope against communication overhead. Smaller domains waste area on local L2, while larger domains suffer excessive traffic.

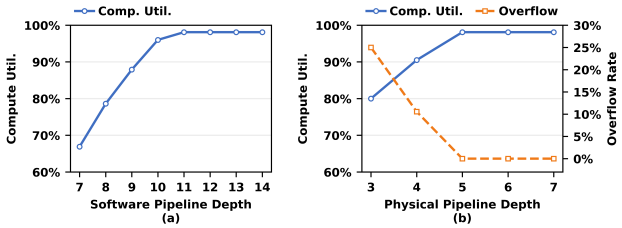


Figure 16: Pipeline depth and physical resource efficiency. (a) ≥12 software stages are needed for >98% utilization, far beyond the 7 stages existing GPUs support. (b) With 12 software stages, only 5 physical slots suffice for full utilization.

degrades compute utilization. The optimal point is d=4, which balances sufficient L2 sharing (effective 64 MB) with manageable communication cost. This validates the domain-partitioned execution design and its configuration in our evaluation.

4.5 Speculative Pipelining Analysis

4.5.1 Pipeline Depth Requirement. To determine the minimum pipeline depth required to hide remote-access latency characterized in Section 2, we vary software pipeline depth from 7 to 14 while providing sufficient physical resources, as shown in Figure 16(a).

Utilization increases monotonically from 68.5% at depth 7 to >98% at depth≥12, demonstrating that at least 12 stages are essential for hiding the remote-access latency. On existing GPUs where pipeline depth is bounded by ⌊SMEM capacity/per-stage size⌋, the maximum achievable depth with 227 KB shared memory on NVIDIA H100 is only 7 stages. This gap between the required 12 stages and the achievable 7 stages underscores the necessity of speculative pipelining: without it, compute utilization is limited to 68.5%, leaving over 30% of the computational potential unutilized.

4.5.2 Physical Resource Sensitivity. To validate that speculative pipelining can achieve the required pipeline depth with limited physical resources, we fix the software pipeline depth at 12 and vary the number of physical pipeline stages, as shown in Figure 16(b).

With only 3 physical slots, speculative pipelining already achieves 79.9% utilization despite a 25.0% overflow rate. As physical stages increase to 5, utilization reaches 98.1% with no overflow. This demonstrates that speculative pipelining enables a Hopper-like shared-memory capacity (7 pipeline slots) to fully sustain 12 software stages, and that shared memory can potentially be reduced to only 5 slots to save area. The result validates the key advantage of speculative pipelining: the software pipeline depth is no longer bounded by physical shared-memory capacity.

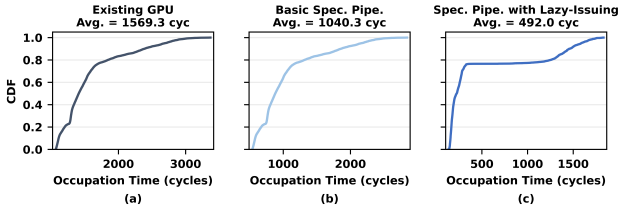


Figure 17: CDF of per-stage shared-memory occupation time. On-demand allocation reduces occupation by 33.7% over conservative allocation. Adding lazy-issuing achieves a 61.6% total reduction, enabling rapid physical-slot turnover.

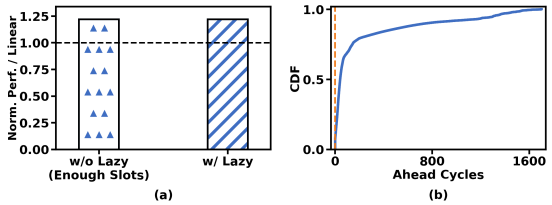


Figure 18: Evaluation of potential negative effect of lazy-issuing delay, showing that it is negligible. (a) Overall performance comparison. (b) Distribution of arrival-ahead cycles.

4.5.3 Occupation Time Reduction. To understand how on-demand allocation and lazy-issuing each contribute to reducing per-stage shared-memory occupation time, the key metric that determines physical-slot turnover rate, we compare three strategies. Figure 17 shows the results. On existing GPUs, the average occupation time is 1569 cycles, dominated by the round-trip latency from load issuance to data arrival. Basic speculative pipelining reduces this to 1040 cycles (33.7% reduction) by deferring resource allocation to first-byte arrival, eliminating the redundant occupation during the issue-to-arrival interval. Adding lazy-issuing further reduces occupation to 492 cycles (68.6% reduction from existing GPUs) by aligning arrival times of requests with different latencies, compressing the first-to-last-byte arrival spread. This tighter occupation window means each physical slot turns over faster, confirming lazy-issuing as essential for sustaining deep pipelines with limited physical resources. The remaining long tail primarily arises during the transient warm-up phase and is handled by warm-up throttling. In steady state, long-occupation stages are rare and occur during congested windows in which subsequent stages also arrive later. Consequently, these long tails do not create sufficient simultaneous slot pressure to cause overflow with five physical stages.

4.5.4 Effect of Lazy-Issuing Delay. We further evaluate whether lazy-issuing defers the slowest response within a stage and thereby affects overall performance, despite reducing occupation time. To isolate this potential delay from the effect of limited physical slots, we compare full HALO against a method without lazy-issuing but with sufficient physical slots. Otherwise, disabling lazy-issuing would introduce overflow. We also examine the detailed latency behavior. Specifically, we measure the slowest response within each stage without lazy-issuing and latency of every delayed request with lazy-issuing. Their difference indicates how many cycles each delayed request arrives before the original slowest response in the same stage, where a negative value indicates that the slowest response is deferred. Figure 18(a) shows that lazy-issuing delay has a negligible negative effect on overall performance with a negligibly 0.11% drop. Figure 18(b) demonstrates that only <0.05% delayed requests defer the slowest response. This is because we only delay requests with short latency and adopt a conservative delay time.

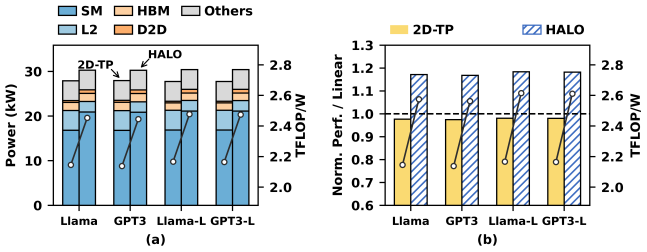


Figure 19: Power evaluation and comparison between HALO and 2D-TP, demonstrating HALO's efficiency. (a) Power consumption and efficiency under iso-frequency setting. (b) Performance and power efficiency under iso-power setting.

4.6 Power Evaluation

Power is a critical design objective for wafer-scale GPUs. We evaluate power consumption using our extended version of AccelWattch [15] that supports Hopper architecture and SoW-X links [37]. We calibrate GPU die model against H100 SXM and then modify its configuration to model our dies. For die-to-die links, we use publicly available SoW-X data [37] to estimate link power.

We first evaluate the iso-area and iso-frequency setting used in our main experiments to characterize the power implications of HALO's iso-area design. As shown in Figure 19(a), HALO consumes 30.3kW on average, only 9.0% more than 2D-TP's 27.8kW. This increase is primarily due to the higher SM count of HALO die, while the smaller L2 offsets part of the cost. Although the increased inter-die traffic volume raises power consumption, its contribution is small relative to those of other components. Because the performance gain exceeds the power increase, HALO improves power efficiency by 14.3% over 2D-TP on average.

We further evaluate an iso-area and iso-power setting by reducing the frequency of HALO until its power matches that of 2D-TP. This setting is important for scenarios with strict power budgets. We find that HALO requires only a modest frequency reduction to 0.965×. Figure 19(b) shows that, even under this iso-power constraint, HALO still achieves a 1.19× average speedup over 2D-TP and improves power efficiency by 20.3% on average. These results demonstrate that HALO's advantage is robust under both iso-area and iso-power constraints.

4.7 Discussion

4.7.1 Diverse Workloads. We evaluate HALO on diverse workloads to validate its generality, including inference (prefill and decode), MoE layer, and RWKV. We adopt Llama 3.1 405B [5] for inference, as in main experiments, DeepSeek-V3 MoE layer [22] for MoE, and RWKV-70B-style recurrent block extrapolated from public RWKV-14B [33] scaling pattern for RWKV. Figure 20 shows that HALO outperforms distributed execution baselines on all these workloads, with speedups of 1.12×, 1.19×, and 1.23×, demonstrating its generality. Speedups for inference and MoE layer are lower than for LLM training because decode provides no data reuse and each expert computation has relatively fine granularity, respectively.

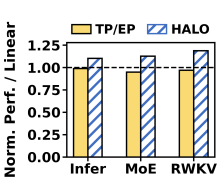


Figure 20: Evaluation across diverse workloads.

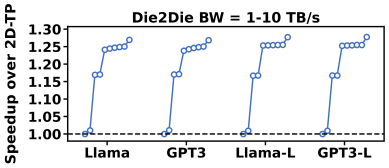


Figure 21: Evaluation across diverse die-to-die bandwidths.

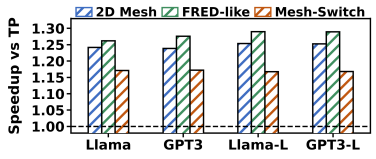


Figure 22: Evaluation across diverse topologies.

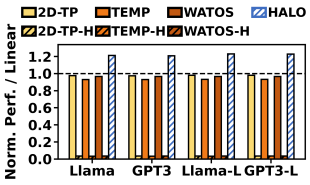


Figure 23: Evaluation across diverse baseline die configurations.

4.7.2 Diverse Die-to-Die Bandwidth. We perform a sensitivity study of Halo across diverse die-to-die bandwidths from 1 to 10 TB/s. For each bandwidth, we conduct a design space exploration to identify the best SM-L2 configuration and L2-sharing domain size. Figure 21 shows that Halo outperforms distributed execution when bandwidth exceeds 1 TB/s, with speedups increasing as bandwidth grows, where speedups in average are 1.01×, 1.24×, and 1.27× for 2, 5, and 10 TB/s. These results demonstrate Halo's performance benefits under high bandwidth provided by wafer-scale integration. It also explains why prior multi-chip research has not explored a Halo-style approach: limited inter-chip bandwidth cannot support cross-die L2 sharing, highlighting the novelty of Halo for wafer-scale GPUs.

4.7.3 Diverse Topology. We perform a topology sensitivity study of Halo using two additional feasible wafer-scale interconnects: FRED-like fabric [35] and mesh-switch fabric [50]. FRED-like fabric is conceptually similar to the high-radix fat-tree NVLink/NV-Switch fabric used in multi-chip systems. We model 32 GPU dies, because switch fabric occupies approximately half of the wafer area [35], and a fabric bandwidth of 5 TB/s. The baseline uses 1D-TP, whereas Halo uses the full wafer as L2-sharing domain. We adapt compute-aligned interleaving accordingly: for $B{=}AW$, W and B are localized to each die's output shard, while A is fine-grain interleaved. For mesh-switch fabric, we model ten 2×2 mesh groups [50] connected through a central switch. Its baseline uses hierarchical TP, with 1D-TP across groups and 2D-TP within each group, whereas Halo confines L2 sharing to each local 2×2 group. Figure 22 shows that Halo improves performance across these diverse topologies, with average speedups of 1.24×, 1.28×, and 1.17×, demonstrating its generality. Its effectiveness on FRED-like fabric, which resembles multi-chip topology, further suggests that Halo could be applied to future multi-chip systems if their fabrics provide comparable bandwidth.

4.7.4 Diverse Baseline Die Configuration. We evaluate Halo against diverse baseline die configurations to validate our baseline selection. In addition to the sweet-spot die used by the distributed execution baselines, we also evaluate these baselines using the Halo die. Figure 23 shows that baselines using the Halo die, denoted by "-H", perform substantially worse than those using the sweet-spot die, dropping by 96.4% in average. Under distributed execution, L2 caches remain isolated, so each die can exploit only its local L2 for local computation. The small local L2 capacity of the Halo die therefore results in poor performance. A fair comparison should provide Halo and each baseline with the same die area and their respective best design configurations, rather than equalizing the number of SMs. The sweet-spot die is therefore a more appropriate configuration for the distributed execution baselines than the Halo die.

4.8 Hardware Overhead

We evaluate hardware overhead of Halo under TSMC 12nm technology [42]. We implement the components in RTL and synthesize them using Synopsys Design Compiler® [39]. Halo introduces three hardware components: the extended TMA address generator from Section 3.2.4, the traffic-class allocator from Section 3.2.4, and the speculative pipelining unit from Section 3.3.4. Per die, TMA extension occupies $3.13{\times}10^{-2}\,\mathrm{mm}^2$, traffic allocator $9.77{\times}10^{-4}\,\mathrm{mm}^2$, and speculative pipelining unit $9.38{\times}10^{-2}\,\mathrm{mm}^2$. The total area cost is $0.126\,\mathrm{mm}^2$ per die, less than 0.02% of die area. This minimal overhead validates that Halo is practical to implement.

5 Related Work

Multi-die and wafer-scale chip architectures. Multi-die GPUs have been extensively studied since MCM-GPU [1], with subsequent work proposing NUMA-aware memory management, scheduling, and caching optimizations [16, 25, 52, 54]. General-purpose wafer-scale GPU architectures [32, 48] adopt single-GPU execution but achieve only sub-linear scaling due to architecture- and workload-unaware traffic management. LLM-centric solutions for training [46, 47], inference [7, 21, 40, 41, 49, 53] adopt distributed execution to achieve linear scaling. Halo departs from both paradigms by *architecturally managing* cross-die remote loads, enabling deliberate L2 sharing that converts memory-system redundancy into additional compute for beyond-linear scaling.

LLM training parallelism. Modern LLM training employs multi-dimensional parallelism: tensor parallelism [38], pipeline parallelism [12, 27], data parallelism [34, 55]. Alpa [56] automates inter- and intra-operator parallelism, and MeshSlice [26] proposes efficient 2D tensor parallelism for 2D topology. These techniques partition computation across independent GPU instances and communicate through collective operations, inherently isolating L2 cache. Halo's introduces a fundamentally different execution model based on SM-direct remote loads within a domain for L2 sharing.

6 Conclusion

This paper presents Halo, a hardware-software co-design framework that enables beyond-linear scaling on wafer-scale GPUs through breaking L2 isolation with architecturally managed SM-direct remote loads. By integrating remote-access scheduling and speculative pipelining, Halo achieves 1.24× average speedup over SOTA distributed execution on LLM training, reaching 98% of the theoretical beyond-linear bound at less than 0.02% die-area overhead.

References

[1] Akhil Arunkumar, Evgeny Bolotin, Benjamin Cho, Ugljesa Milic, Eiman Ebrahimi, Oreste Villa, Aamer Jaleel, Carole-Jean Wu, and David Nellans. 2017. MCM-GPU: Multi-chip-module GPUs for continued performance scalability. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 320–332.

[2] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

[4] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[5] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[6] Haoming Guo, Shengbin Cao, Lei Li, and Xiaofeng Zhang. 2022. A review on the mainstream through-silicon via etching methods. *Materials Science in Semiconductor Processing* 137 (2022), 106182.

[7] Congjie He, Yeqi Huang, Pei Mu, Ziming Miao, Jilong Xue, Lingxiao Ma, Fan Yang, and Luo Mai. 2025. {WaferLLM}: Large Language Model Inference at Wafer Scale. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 257–273.

[8] SY Hou, Chien Hsun Lee, Tsung-Ding Wang, Hao Cheng Hou, and Hsieh-Pin Hu. 2023. Supercarrier redistribution layers to realize ultra large 2.5 D wafer scale packaging by CoWoS. In *2023 IEEE 73rd Electronic Components and Technology Conference (ECTC)*. IEEE, 510–514.

[9] Chih-Hao Hsia, SungHo Park, Soichi Watanabe, Marco Arnold, Zaid El-Mekki, Martine Delande, Ehsan Shafahian, Punith KMK Gowda, Herbert Struyf, and Aleksandar Radisic. 2024. Wafer-Level Electrochemical Deposition and Processing of Nanotwinned Cu RDL. In *2024 IEEE International Interconnect Technology Conference (IITC)*. IEEE, 1–3.

[10] Yu-Chen Hu, Yu-Min Liang, Hsieh-Pin Hu, Chia-Yen Tan, Chih-Ta Shen, Chien-Hsun Lee, and SY Hou. 2023. CoWoS architecture evolution for next generation HPC on 2.5 D system in package. In *2023 IEEE 73rd Electronic Components and Technology Conference (ECTC)*. IEEE, 1022–1026.

[11] PK Huang, CY Lu, WH Wei, Christine Chiu, KC Ting, Clark Hu, CH Tsai, SY Hou, WC Chiou, CT Wang, et al. 2021. Wafer level system integration of the fifth generation CoWoS®-S with high performance Si interposer at 2500 mm2. In *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*. IEEE, 101–104.

[12] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).

[13] Nan Jiang, Daniel U Becker, George Michelogiannakis, James Balfour, Brian Towles, David E Shaw, John Kim, and William J Dally. 2013. A detailed and flexible cycle-accurate network-on-chip simulator. In *2013 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 86–96.

[14] Binbin Jiao, Jingping Qiao, Shiqi Jia, Ruiwen Liu, Xueyong Wei, Shichang Yun, Yanmei Kong, Yuxin Ye, Xiangbin Du, Lihang Yu, et al. 2024. Low stress TSV arrays for high-density interconnection. *Engineering* 38 (2024), 201–208.

[15] Vijay Kandiah, Scott Peverelle, Mahmoud Khairy, Junrui Pan, Amogh Manjunath, Timothy G Rogers, Tor M Aamodt, and Nikos Hardavellas. 2021. AccelWattch: A power modeling framework for modern GPUs. In *MICRO-54: 54th Annual IEEE/ACM International symposium on microarchitecture*. 738–753.

[16] Mahmoud Khairy, Vadim Nikiforov, David Nellans, and Timothy G Rogers. 2020. Locality-centric data and threadblock management for massive GPUs. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1022–1036.

[17] Mahmoud Khairy, Zhesheng Shen, Tor M Aamodt, and Timothy G Rogers. 2020. Accel-Sim: An extensible simulation framework for validated GPU modeling. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 473–486.

[18] John H Lau. 2023. Recent advances and trends in multiple system and heterogeneous integration with TSV interposers. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 13, 1 (2023), 3–25.

[19] John H Lau, Gary Chang-Fu Chen, Jones Yu-Cheng Huang, Ricky Tsun-Sheng Chou, Channing Cheng-Lin Yang, Hsing-Ning Liu, and Tzyy-Jang Tseng. 2021. Hybrid substrate by fan-out RDL-first panel-level packaging. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 11, 8 (2021), 1301–1309.

[20] John H Lau, Cheng-Ta Ko, Kai-Ming Yang, Chia-Yu Peng, Tim Xia, Puru Bruce Lin, JJ Chen, Patrick Po-Chun Huang, Hsing-Ning Liu, Tzyy-Jang Tseng, et al. 2020. Panel-level fan-out RDL-first packaging for heterogeneous integration. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 10, 7 (2020), 1125–1137.

[21] Sean Lie. 2024. Wafer-scale AI: GPU impossible performance. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE Computer Society, 1–71.

[22] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[23] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 10012–10022.

[24] Gabriel H Loh, Yuan Xie, and Bryan Black. 2007. Processor design in 3D die-stacking technologies. *Ieee Micro* 27, 3 (2007), 31–48.

[25] Ugljesa Milic, Oreste Villa, Evgeny Bolotin, Akhil Arunkumar, Eiman Ebrahimi, Aamer Jaleel, Alex Ramirez, and David Nellans. 2017. Beyond the socket: NUMA-aware GPUs. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. 123–135.

[26] Hyoungwook Nam, Gerasimos Gerogiannis, and Josep Torrellas. 2025. MeshSlice: Efficient 2D Tensor Parallelism for Distributed DNN Training. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 821–834.

[27] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.

[28] Mark Neisser. 2021. International roadmap for devices and systems lithography roadmap. *Journal of Micro/Nanopatterning, Materials, and Metrology* 20, 4 (2021), 044601–044601.

[29] NVIDIA. 2022. NVIDIA H100 Tensor Core GPU. *https://www.nvidia.com/en-us/data-center/h100* (2022).

[30] NVIDIA. 2024. NVIDIA Blackwell Architecture Technical Brief. *https://resources.nvidia.com/en-us-blackwell-architecture* (2024).

[31] NVIDIA. 2024. NVIDIA GB200 NVL72. *https://www.nvidia.com/en-us/data-center/gb200-nvl72/* (2024).

[32] Saptadeep Pal, Daniel Petrisko, Matthew Tomei, Puneet Gupta, Subramanian S Iyer, and Rakesh Kumar. 2019. Architecting waferscale processors-a gpu case study. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 250–263.

[33] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Leon Derczynski, et al. 2023. Rwkv: Reinventing rnns for the transformer era. In *Findings of the association for computational linguistics: EMNLP 2023*. 14048–14077.

[34] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[35] Saeed Rashidi, William Won, Sudarshan Srinivasan, Puneet Gupta, and Tushar Krishna. 2025. FRED: A Wafer-scale Fabric for 3D Parallel DNN Training. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 34–48.

[36] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, et al. 2019. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*. 14–27.

[37] Po-Chang Shih, An-Jhih Su, King-Ho Tam, Tze-Chiang Huang, Kris Chuang, and John Yeh. 2025. SoW-X: A novel system-on-wafer technology for next generation AI server application. In *2025 IEEE 75th Electronic Components and Technology Conference (ECTC)*. IEEE, 1–6.

[38] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[39] Synopsys. 2021. Design Compiler® RTL Synthesis. *https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-nxt.html* (2021).

[40] Emil Talpes, Douglas Williams, and Debjit Das Sarma. 2022. Dojo: The microarchitecture of tesla's exa-scale computer. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE, 1–28.

[41] Xinru Tang, Jingxiang Hou, Dingcheng Jiang, Taiquan Wei, Jiaxin Liu, Jinyi Deng, Huizheng Wang, Qize Yang, Haoran Shang, Chao Li, et al. 2026. MoEntwine: Unleashing the potential of wafer-scale chips for large-scale expert parallel inference. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–15.

[42] TSMC. 2017. TSMC 16nm and 12nm process technologies. *https://www.tsmc.com/english/dedicatedFoundry/technology/logic/l_16_12nm* (2017).

[43] TSMC. 2025. TSMC's Chip on Wafer on Substrate. *https://3dfabric.tsmc.com/english/dedicatedFoundry/technology/cowos.htm*

(2025).
[44] Walker J Turner, John W Poulton, John M Wilson, Xi Chen, Stephen G Tell, Matthew Fojtik, Thomas H Greer, Brian Zimmer, Sanquan Song, Nikola Nedovic, et al. 2018. Ground-referenced signaling for intra-chip and short-reach chip-to-chip interconnects. In *2018 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 1–8.
[45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[46] Huizheng Wang, Zichuan Wang, Hongbin Wang, Jingxiang Hou, Taiquan Wei, Chao Li, Yang Hu, and Shouyi Yin. 2026. WATOS: Efficient LLM training strategies and architecture co-exploration for wafer-scale chip. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–19.
[47] Huizheng Wang, Taiquan Wei, Zichuan Wang, Dingcheng Jiang, Qize Yang, Jiaxin Liu, Jingxiang Hou, Chao Li, Jinyi Deng, Yang Hu, et al. 2026. TEMP: A memory efficient physical-aware tensor partition-mapping framework on wafer-scale chips. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–18.
[48] Daoxuan Xu, Ying Li, Yuwei Sun, Jie Ren, and Yifan Sun. 2026. HDPAT: Hierarchical Distributed Page Address Translation for Wafer-Scale GPUs. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–12.
[49] Zheng Xu, Dehao Kong, Jiaxin Liu, Jinxi Li, Jingxiang Hou, Xu Dai, Chao Li, Shaojun Wei, Yang Hu, and Shouyi Yin. 2025. Wsc-llm: Efficient llm service and architecture co-exploration for wafer-scale chips. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1–17.
[50] Qize Yang, Taiquan Wei, Sihan Guan, Chengran Li, Haoran Shang, Jinyi Deng, Huizheng Wang, Chao Li, Lei Wang, Yan Zhang, et al. 2025. Pd constraint-aware physical/logical topology co-design for network on wafer. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 49–64.
[51] Sheng-Fan Yang, Wei-Chiao Wang, Yi-Tzeng Lin, Chih-Chiang Hung, Hao-Yu Tung, and Justin Hsieh. 2024. Signal integrity designs at organic interposer CoWoS-R for HBM3-9.2 Gbps high speed interconnection of 2.5 D-IC chiplets integration. In *2024 IEEE 74th Electronic Components and Technology Conference (ECTC)*. IEEE, 1098–1103.
[52] Vinson Young, Aamer Jaleel, Evgeny Bolotin, Eiman Ebrahimi, David Nellans, and Oreste Villa. 2018. Combining HW/SW mechanisms to improve NUMA performance of multi-GPU systems. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 339–351.
[53] Xingmao Yu, Dingcheng Jiang, Jinyi Deng, Jingyao Liu, Chao Li, Shouyi Yin, and Yang Hu. 2025. Cramming a data center into one cabinet, a co-exploration of computing and hardware architecture of waferscale chip. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 631–645.
[54] Shiqing Zhang, Mahmood Naderan-Tahan, Magnus Jahre, and Lieven Eeckhout. 2023. SAC: Sharing-aware caching in multi-chip GPUs. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
[55] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).
[56] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.